

# OpenClaw Peripheral Hardware Configuration

---

## OpenClaw Peripheral Hardware Configuration

1. Course Overview
2. Hardware Default Configuration File Description
  - 2.1 Configuration File Location
  - 2.2 Current Configuration File Coverage
3. DHT Current Configuration Features
  - 3.1 Verification Method
4. RGB Default Configuration
  - 4.1 Verification Method
5. OpenClaw Firmware Serial Port Default Configuration (Servo Verification)
  - 5.1 Current Default Values
  - 5.2 Modification Example
  - 5.3 Verification Method
6. OLED Default Configuration
  - 6.1 Current Default Values
  - 6.2 Scenarios Suitable for Modification
  - 6.3 Common Modifications
    - Changing Resolution
    - Changing I2C Address
    - Changing Rotation Direction
  - 6.4 Verification Method
7. Camera Hardware Configuration
  - 7.1 Configuration
  - 7.2 Checking Ports
  - 7.3 Photo Verification Method

## 1. Course Overview

---

This course introduces the default hardware-related configuration files used before operating OpenClaw, with a focus on how to modify the underlying default parameters based on the actual peripherals you have, and complete basic verification.

- Use underlying commands to verify whether modifications have taken effect

## 2. Hardware Default Configuration File Description

---

### 2.1 Configuration File Location

OpenClaw's current hardware default configuration file is located at:

```
/home/jetson/hardware_hal_demo/hardware_config.py
```

If you are only following the accompanying tutorial and using the bundled factory hardware, you usually do not need to modify it.

## 2.2 Current Configuration File Coverage

`hardware_config.py` mainly contains three types of default configurations:

- OpenClaw firmware serial port default parameters
- OLED default parameters
- CSI camera or USB camera

These default values are read by underlying command-line tools, such as:

- `rgb`
- `oled`
- `openclaw_adapter`
- `CAMERA_DEFAULTS`

Please note:

- DHT and RGB are currently driven directly by the OpenClaw firmware
- OLED is driven by the motherboard's I2C pins
- CSI camera/USB camera requires checking the hardware connection

## 3. DHT Current Configuration Features

### 3.1 Verification Method

```
dht      #Get sensor data
dht_oled #Display sensor data on OLED
```

Normal terminal output should show the OLED displaying temperature and humidity data.

```
jetson@yahboom:~$ dht
{"ok": true, "sensor": "dht11", "temperature_c": 27.4, "temperature_f": 81.3, "humidity": 59.0, "attempts": 1}
jetson@yahboom:~$ dht_oled
{"ok": true, "code": "ok", "stage": "completed", "message": "温湿度已显示到 OLED", "reading": {"ok": true, "sensor": "dht11", "temperature_c": 27.5, "temperature_f": 81.5, "humidity": 55.0, "attempts": 1}, "oled": {"title": "DHT11", "text": "Temp:27.5C\nHum:55.0%", "lines": ["DHT11", "Temp:27.5C", "Hum:55.0%"], "dry_run": false, "i2c_bus": "auto", "render_title": "DHT11", "render_text": "Temp:27.5C\nHum:55.0%"}}
```

## 4. RGB Default Configuration

### 4.1 Verification Method

After saving, run the following in the terminal:

```
rgb color --color red #All lights turn red
rgb effect --effect breathe --color blue --speed 5 #Blue breathing light
```

Normal terminal output should show the RGB lights displaying correctly.

```
jetson@yahboom:~$ rgb color --color red
{"ok": true, "code": "ok", "message": "RGB 颜色已设置", "details": {"index": 255, "r": 255, "g": 0, "b": 0}}
jetson@yahboom:~$ rgb effect --effect breathe --color blue --speed 5
{"ok": true, "code": "ok", "message": "RGB 特效已设置: breathe", "details": {"effect": "breathe", "speed": 5, "color": "blue"}}
```

## 5. OpenClaw Firmware Serial Port Default Configuration (Servo Verification)

## 5.1 Current Default Values

The current OpenClaw firmware serial port default parameters are:

- `port="/dev/ttyTHS1"`
- `baudrate=115200`

## 5.2 Modification Example

Edit the configuration file:

```
/home/jetson/hardware_hal_demo/hardware_config.py
```

For Jetson Orin NX, change the values as follows:

```
"port": "/dev/ttyTHS1",  
"baudrate": 115200,
```

## 5.3 Verification Method

After saving, we recommend first testing with a servo or RGB using any underlying command, for example:

```
rgb color --color green
```

Or:

```
servo call-tool Servo1 --action center #Servo 1 centers to 90 degrees  
# All servos return to center position  
servo call-tool ServoAll --action set_all --angle1 90 --angle2 90 --angle3 90 --  
angle4 90
```

If the commands execute successfully and control the peripherals properly, the serial port link is working.

```
jetson@yahboom:~$ servo call-tool Servo1 --action center  
{  
  "ok": true, "code": "ok", "message": "done", "device_id": "servo_1", "action": "center", "request_id": "", "state":  
    {  
      "angle": 90.0, "attached": true, "last_angle": 90.0, "min_angle": 0.0, "max_angle": 180.0, "telemetry": {  
        "mode":  
        "direct", "driver": "openclaw_servo", "servo_id": 1, "min_angle": 0.0, "max_angle": 180.0, "action": "set_angle", "p  
        arams": {}, "retryable": false  
      }  
    }  
}  
jetson@yahboom:~$ servo call-tool ServoAll --action set_all --angle1 90 --angle2 90 --angle3 90 --angle4 90  
{  
  "ok": true, "code": "ok", "message": "done", "device_id": "all", "action": "set_all", "request_id": "", "state": {  
    "angles": [90, 90, 90, 90]}, "telemetry": {  
      "mode": "direct", "params": {  
        "angle1": 90.0, "angle2": 90.0, "angle3": 90.  
        0, "angle4": 90.0  
      }  
    }  
}  
jetson@yahboom:~$
```

## 6. OLED Default Configuration

### 6.1 Current Default Values

The current OLED default parameters mainly include:

- `width=128`
- `height=32`
- `interface="i2c"`
- `i2c_bus="auto"`
- `i2c_bus_candidates=[1, 0, 7, 8]`

- `i2c_address=0x3C`
- `rotate=0`
- `invert=False`
- `contrast=255`

## 6.2 Scenarios Suitable for Modification

It is recommended to modify the OLED defaults in the following situations:

- You are using an OLED with a different resolution
- The I2C address is not `0x3C`
- The screen orientation needs to be flipped

## 6.3 Common Modifications

### Changing Resolution

For example, change:

```
"width": 128,  
"height": 32,
```

To:

```
"width": 128,  
"height": 64,
```

### Changing I2C Address

For example, change:

```
"i2c_address": 0x3C,
```

To:

```
"i2c_address": 0x3D,
```

### Changing Rotation Direction

For example, change:

```
"rotate": 0,
```

To:

```
"rotate": 180,
```

## 6.4 Verification Method

After saving, run the following in the terminal:

```
oled text --text 'Hello OpenClaw'
oled clear
```

If the OLED can display text and clear the screen normally, the configuration has taken effect.

```
jetson@yahboom:~$ oled text --text 'Hello OpenClaw'
{"ok": true, "code": "ok", "mode": "text", "message": "文字已显示到 OLED", "details": {"title": "", "text": "Hello OpenClaw", "lines": ["Hello OpenClaw"], "dry_run": false}}
jetson@yahboom:~$ oled clear
{"ok": true, "code": "ok", "mode": "clear", "message": "OLED 已清屏", "details": {"dry_run": false}}
jetson@yahboom:~$
```

## 7. Camera Hardware Configuration

### 7.1 Configuration

Prerequisite: ensure that a CSI camera is connected. The current CSI camera link works mainly through the following two entry points:

If using a USB camera, you can modify the configuration in this file:

```
/home/jetson/hardware_hal_demo/hardware_config.py
```

```
# Camera default configuration
# backend: "csi" uses Jetson CSI camera; "usb" uses USB camera
CAMERA_DEFAULTS = {
    "backend": "usb",
    "usb_device": "/dev/video0",
    "usb_format": "auto",
    "width": 1280,
    "height": 720,
    "framerate": 30,
    "flip_method": 2,
    "sensor_id": 0,
}
```

Camera CLI command path:

```
/home/jetson/hardware_hal_demo/cli/seewhat
```

### 7.2 Checking Ports

Enter in the terminal:

```
ls /dev/video*
```

Normally, a CSI camera should show one device, video0, and a USB camera should show video0 and video1 devices.

```
jetson@yahboom:~$ ls /dev/video*
/dev/video0
jetson@yahboom:~$
```

## 7.3 Photo Verification Method

Enter the following command in the terminal to test the photo capture:

```
seewhat call-tool Seewhat
```

```
jetson@yahboom:~$ seewhat call-tool Seewhat
{"ok": true, "code": "ok", "message": "capture complete", "image": "/home/jetson/hardware_hal_demo/cli/latest.jpg",
"image_path": "/home/jetson/hardware_hal_demo/cli/latest.jpg", "device": "jetson-csi", "backend": "gstreamer", "width":
1280, "height": 720, "framerate": 30, "flip_method": 2, "sensor_id": 0, "size_bytes": 53405, "elapsed_ms": 1963,
"request_id": "", "saved_name": "latest.jpg", "pipeline": "nvarguscamerasrc sensor-id=0 ! video/x-raw(memory:NVMM),
width=1280, height=720, framerate=30/1 ! nvvidconv flip-method=2 ! video/x-raw(memory:NVMM), format=NV12 ! nvjpegen
c ! multifilesink location=/home/jetson/hardware_hal_demo/cli/latest.capture-%05d.jpg max-files=1", "inherit_display
_env": false, "display_env_before_capture": {}, "display_env_removed": {}}
```

If the photo is captured successfully, you should see the output file in the following directory:

```
/home/jetson/.openclaw/workspace/photos/latest.jpg
```



